

# IT5008: Tutorial 4 — Aggregate and Nested Queries

Pratik Karmakar

School of Computing,  
National University of Singapore

AY26/27 S1



# The Book Exchange Schema (Recap)

Same NUN Book Exchange scenario and schema as Tutorial 3.

- **department**(department, faculty)
- **student**(name, email, year, department (*FK*), graduate)
- **book**(title, format, pages, language, author, publisher, pubdate, isbn10 (*unique*), ISBN13)
- **copy**(owner, book, copy) — FKs to student and book
- **loan**(borrower, owner, book, copy, borrowed, returned)

## New this tutorial

**Aggregate** queries (COUNT, AVG, STDDEV\_POP, GROUP BY, HAVING) and **nested** queries (IN, NOT IN, EXISTS, NOT EXISTS, ALL) — including **universal quantification** written as a double negation.

# Preliminary: Aggregate & Nested Queries

*Not discussed in class — attempt before the tutorial, since later questions build on these.*

## 1. Aggregate Queries.

- (a) How many loans involve an owner and a borrower from the *same* department?
- (b) For each faculty, print the number of loans that involve an owner and a borrower from *this* faculty.

## 2. Nested Queries.

- (a) Print the titles of the different books that have *never* been borrowed. Use a nested query.
- (b) Print the emails and names of the different students who borrowed *all* the books authored by 'Adam Smith'.

## 1(a). Loans Within the Same Department

Join loan to student **twice** — once for the owner's department, once for the borrower's — then count.

```
1 SELECT COUNT(*)
2 FROM loan l, student s1, student s2
3 WHERE l.owner = s1.email
4       AND l.borrower = s2.email
5       AND s1.department = s2.department;
```

## 1(b). Same-Faculty Loans, per Faculty

Add department twice more to resolve each side's faculty, then GROUP BY the (shared) faculty.

```
1 SELECT d1.faculty, COUNT(*)
2 FROM loan l, student s1, student s2, department d1, department d2
3 WHERE l.owner = s1.email
4     AND l.borrower = s2.email
5     AND s1.department = d1.department
6     AND s2.department = d2.department
7     AND d1.faculty = d2.faculty
8 GROUP BY d1.faculty;
```

## 2(a). Books Never Borrowed — via Nested Query

A book is “never borrowed” iff its ISBN-13 never shows up in loan.

```
1 SELECT b.title
2 FROM book b
3 WHERE b.ISBN13 NOT IN (
4     SELECT l.book
5     FROM loan l
6 );
```

## 2(a). Two Tempting but Wrong Answers

**Wrong #1** — joining through copy silently drops books that have *no copy at all*:

```
1 SELECT b.title
2 FROM book b, copy c
3 WHERE b.ISBN13 = c.book
4     AND b.ISBN13 NOT IN (SELECT l.book FROM loan l);
```

**Wrong #2** — this checks whether a *copy* was borrowed, not whether the *book* was:

```
1 SELECT b.title
2 FROM book b, copy c
3 WHERE b.ISBN13 = c.book
4     AND NOT EXISTS (
5         SELECT * FROM loan l
6         WHERE l.book = c.book AND l.copy = c.copy
7     );
```

Both need a copy to exist; the question is about the **book**.

## 2(b). Borrowed *All* of Adam Smith's Books

**Universal quantification:** “for every Adam Smith book, there’s no case of *s* not borrowing it.”

```
1 SELECT s.email, s.name
2 FROM student s
3 WHERE NOT EXISTS (
4     SELECT * FROM book b
5     WHERE b.authors = 'Adam Smith'
6         AND NOT EXISTS (
7             SELECT * FROM loan l
8             WHERE l.book = b.ISBN13 AND l.borrower = s.email
9         )
10 );
```

## 2(b). Borrowed *All* of Adam Smith's Books

| Student | Truth Value |
|---------|-------------|
| s1      | T           |
| s2      | T           |
| s3      | F           |
| s4      | F           |
| s5      | T           |
| s6      | F           |
| s7      | T           |

Students who have borrowed all books by Adam Smith - Students who have truth value T in the table shown - But how do we assign the truth values?

**The logical statement to evaluate:**

If **there is a book by Adam Smith**, **the student has borrowed it.**

P

Q

**The Boolean Logic Form:**

$P \rightarrow Q$  (P implies Q)  $\equiv \neg P \vee Q$

| P | Q | $P \rightarrow Q$ | $\neg P \vee Q$ |
|---|---|-------------------|-----------------|
| F | F | T                 | T               |
| F | T | T                 | T               |
| T | F | F                 | F               |
| T | T | T                 | T               |

## 2(b). Borrowed *All* of Adam Smith's Books

```

SELECT s.email, s.name
FROM student s
WHERE NOT EXISTS (
    SELECT *
    FROM book b
    WHERE authors = 'Adam Smith'
    AND NOT EXISTS (
        SELECT *
        FROM loan l
        WHERE l.book = b.isbn13
        AND l.borrower = s.email));

```

Diagram illustrating the query structure and its logical components:

- NOT Gate ( $\neg$ )**: Points to the `NOT EXISTS` operator in the main query.
- A book by Adam Smith exists (P)**: Points to the inner query:
 

```

      SELECT *
      FROM book b
      WHERE authors = 'Adam Smith'
      
```
- AND Gate ( $\wedge$ )**: Points to the `AND NOT EXISTS` operator in the nested query.
- The student borrows the book (Q)**: Points to the innermost query:
 

```

      SELECT *
      FROM loan l
      WHERE l.book = b.isbn13
      AND l.borrower = s.email);
      
```

$$\neg(P \wedge \neg Q) \equiv \neg P \vee \neg(\neg Q) \equiv \neg P \vee Q \equiv P \rightarrow Q$$

## Discussion: Aggregate, Nested & Nested-Aggregate

*To be discussed in class — attempt before the tutorial; you may be called on to present.*

- 3. Aggregate Queries.** What are the average and the standard deviation of the duration of a loan, in days?
- 4. Nested Queries.** Print the names of the different students who own a copy of a book that they have *never* lent to anybody.
- 5. Nested Aggregate Queries.** For each department, print the names of the students who lent the most.

### 3. Average & Standard Deviation of Loan Duration

A loan still out has no fixed duration yet — same COALESCE trick as Tutorial 3's 1(c). CEIL rounds up to whole days.

```
1 SELECT CEIL(AVG(COALESCE(l.returned, CURRENT_DATE) - l.borrowed +  
2     1)),  
3     CEIL(STDDEV_POP(COALESCE(l.returned, CURRENT_DATE)  
4     - l.borrowed + 1))  
FROM loan l;
```

**Keep it simple.** A nested-query alternative exists, but a single-table aggregate is the more natural fit here.

## 4. Owns a Copy, Never Lent It

**Inner query:** owners of a copy with no matching loan at all (i.e., that exact copy was never lent).

```
1 SELECT s.name
2 FROM student s
3 WHERE s.email IN (
4     SELECT c.owner
5     FROM copy c
6     WHERE NOT EXISTS (
7         SELECT * FROM loan l
8         WHERE l.owner = c.owner AND l.book = c.book AND l.copy = c.
9         copy
10    );
```

We reduce the inner result to a **set of owners** first, *then* look up names.

## 4. Why Not Just Join Student to Copy?

```
1 SELECT s.name
2 FROM student s, copy c
3 WHERE s.email = c.owner
4     AND NOT EXISTS (
5         SELECT * FROM loan l
6         WHERE l.owner = c.owner AND l.book = c.book AND l.copy = c.
7             copy
8     );
```

**Incorrect.** A student with *several* never-lent copies would have their name printed once per copy — and we'd have no way to tell that apart from two different students who happen to share a name.

## 5. Who Lent the Most, per Department

Count each owner's loans, then keep only the rows whose count is **at least** every other same-department count.

```
1 SELECT s.department, s.name, COUNT(*)
2 FROM student s, loan l
3 WHERE l.owner = s.email
4 GROUP BY s.department, s.email, s.name
5 HAVING COUNT(*) >= ALL (
6     SELECT COUNT(*)
7     FROM student s1, loan l1
8     WHERE l1.owner = s1.email
9     AND s.department = s1.department
10    GROUP BY s1.email
11 );
```

**Ties are real:** the Language department has *multiple* top lenders — exactly why one should avoid TOP N / LIMIT queries here.

## 5. The Zero-Loan Edge Case

Suppose a new department, *Undecidable Computations*, has students who have **never lent** a book.

- Every one of them lent **0** books — which is the maximum in that department.
- So they should **all** be printed, not none of them.
- But our query above never produces a 0-count row for them at all: the FROM student s, loan l join silently **drops** students with no loans.

**Fix:** an OUTER JOIN from student to loan, with CASE WHEN / ISNULL to turn “no matching loan” into a count of 0.

**Also note:** we group by s.name purely to be able to SELECT it, even though s.email already determines it uniquely. PostgreSQL relaxes this rule — but relying on that hurts portability.

## Guidelines & Common Pitfalls

- **Simple first.** If a single-table aggregate answers the question (Q3), prefer it over a nested rewrite.
- **Match the grain of the question.** “Books never borrowed” is about book, not copy — don’t join in a table that changes what “never” ranges over (2(a)).
- **Reduce, then look up.** For nested IN/EXISTS queries producing names, find the qualifying *keys* first (in a subquery), then join for display columns — joining too early can duplicate or misattribute rows (Q4).
- **Avoid TOP N / LIMIT for “the most”.** Ties are common; HAVING COUNT(\*) >= ALL (...) naturally keeps every tied winner (Q5).
- **Mind the zero case.** A student/department with no matching rows can still be the correct answer — OUTER JOIN + CASE/ISNULL, not an inner join, is what surfaces it (Q5).

Questions?

Drop a mail at: [pratik.karmakar@u.nus.edu](mailto:pratik.karmakar@u.nus.edu)